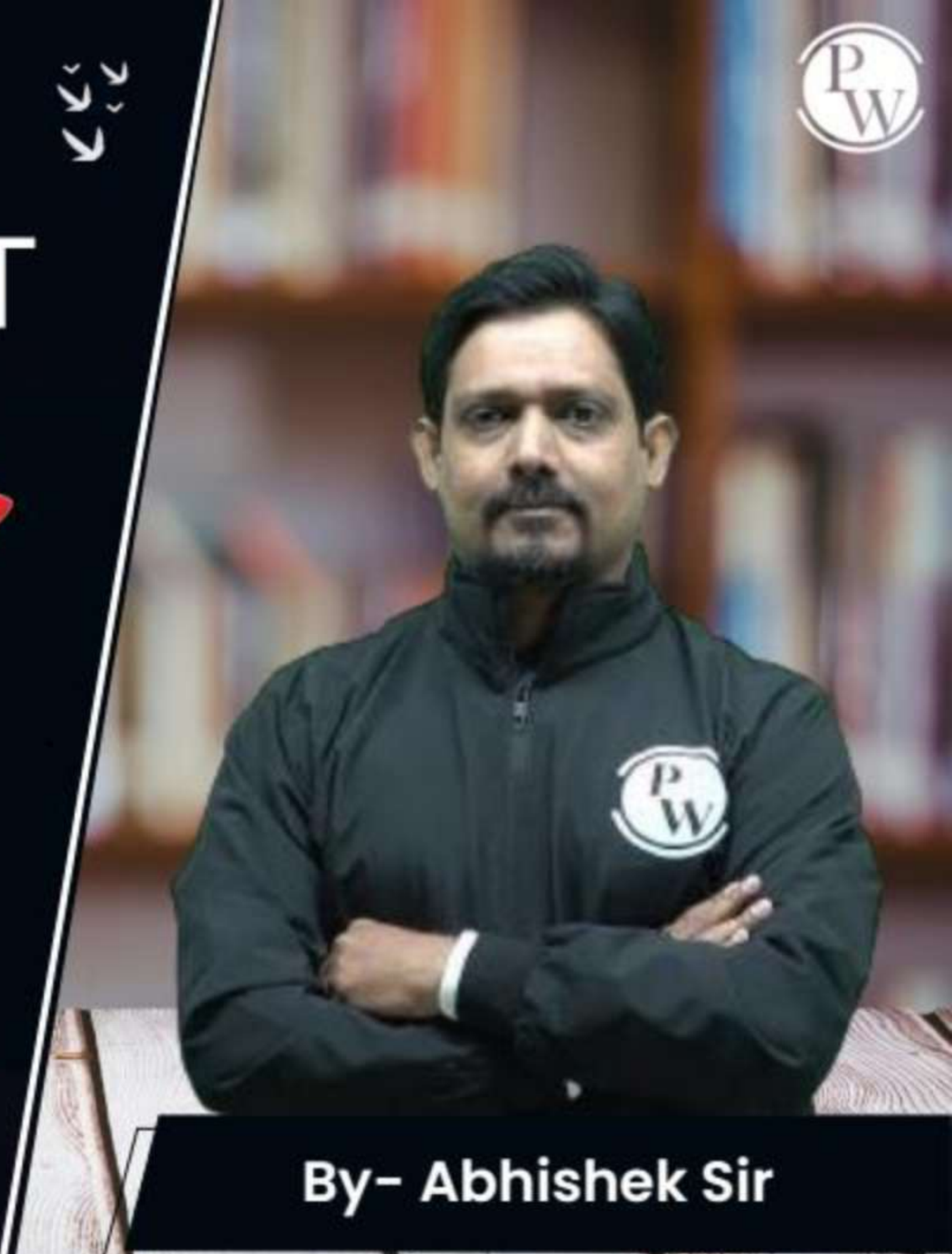
Computer Science & IT



C programming

Array & Pointers

Lecture No. 01



By- Abhishek Sir

Recap of Previous Lecture



- Topic — TOH
- Topic — Counting function → partial Recursion
- Topic — Computation of value Bottom up
  Smaller → to Larger
- Topic
- Topic

Slide

Topics to be Covered



- Topic — Extern & Register
- Topic — panter
- Topic — Call by reference
- Topic
- Topic

Slide



Extern & Register

Storage class Register (Hardware)

Memory ⟶

Register

Cache Memory

Main Memory

Magnatic Disc

Size Increase

Speed decrease

Slide



Extern & Register

Register fastest Access memory (Inside the processor)

No. Register are limit.

In big Application program Some variable are frequently Access. Hence compiler request processor to allocate register for those variable

Slide



Extern & Register

1 only Local variable can be declared as register

2 Register storage class, scope, Lifetime, Initial value is same as auto storage class.

3 In this we can't use & operator. & operator Address of operator

Slide



Extern & Register

Extern storage

1 Declaration & definition

Declaration Name of variable & its attribute

Definition is associating a variable to its memory location.

```
extern int a;
```

is only declaration of variable, No memory is allocated.

Slide



Extern & Register

1. External refer only to global variable
2. if variable declared as extern then No memory is allocated
3. external variable extends scope of variable.
   1. out side of block
   2. outside main (global)
   3. after main
   4. In other files
4. If variable definition is Not found then it leads to Linker error

Slide



Pointers

pointer : pointer is a variable that holds Address of another variable of same type.

```
int a = 10
```



↑ Assumed

to access address of a variable. we need to use & a.

Slide

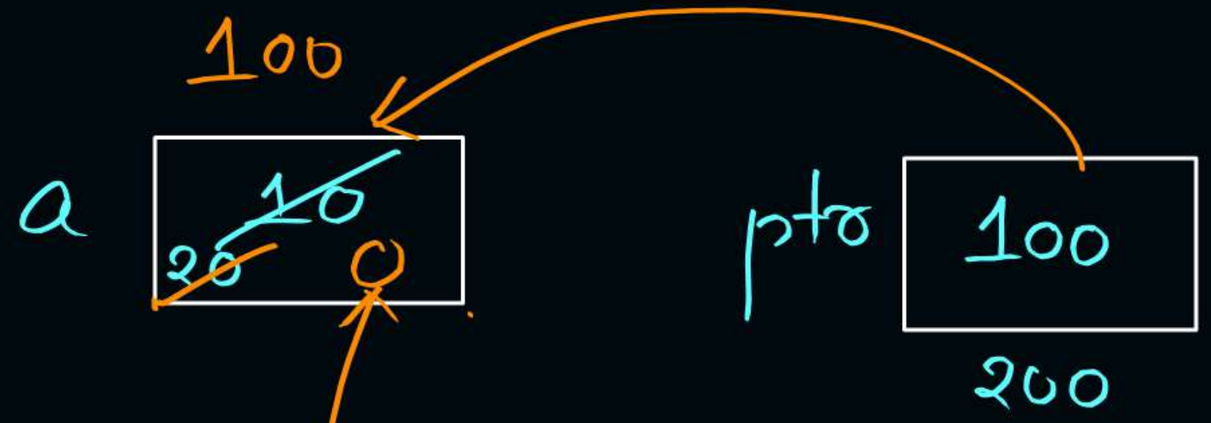


Pointers

pointer variable declaration int * ptr;

ptr is pointer to an integer

pointer assignment : p = &a;



ptr = &a;

a = a + 10

ptr is 100

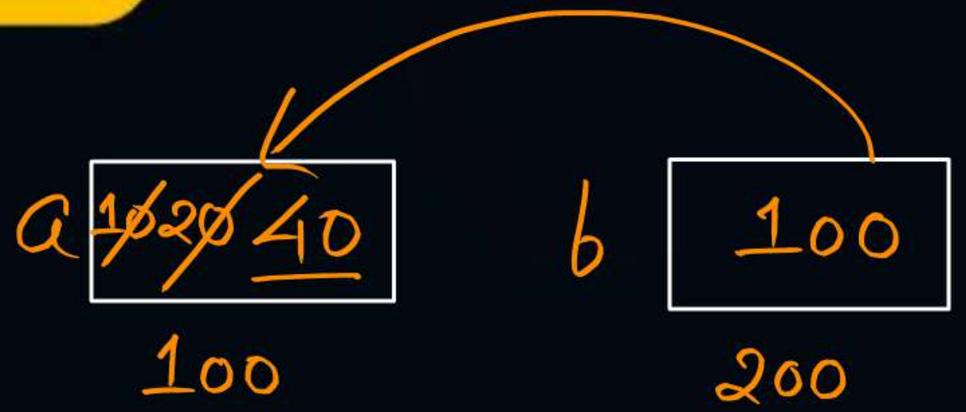
*ptr = 20

*ptr = *ptr - 20;



Pointers

```
#include<stdio.h>
int main (){
        int a, *b;
        a = 10;
        b = &a;
        a =a+10;
        *b = *b+20;
        printf("%d", a);
        return 0;
}
```



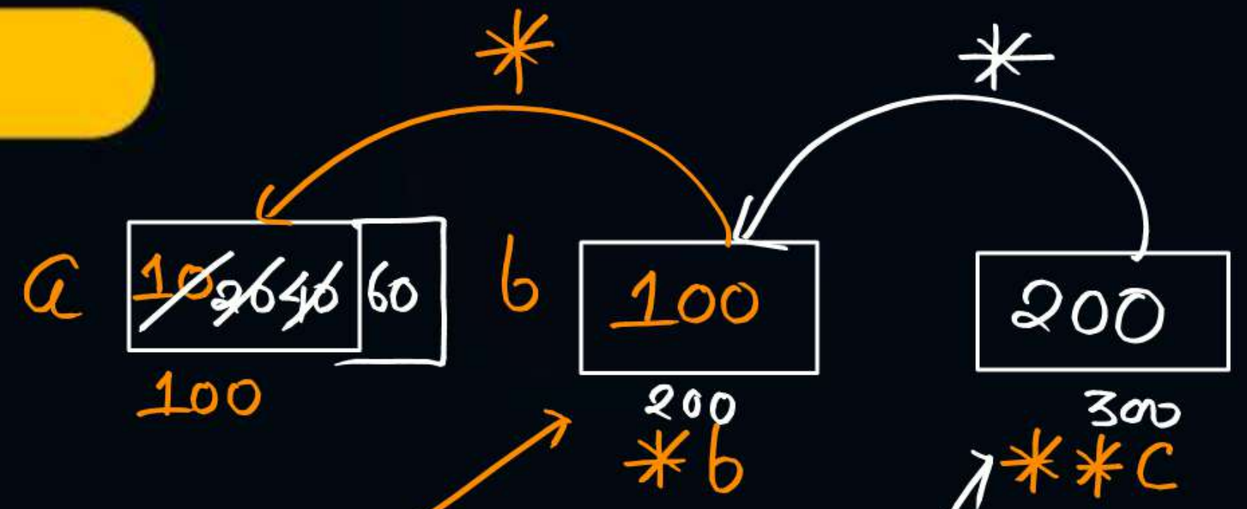
← 40

Slide

Pointers



```
#include<stdio.h>
int main (){
        int a, *b, **c;
        a = 10;
        b = &a;
        c = &b;
        a =a+10;
        *b = *b+20;
        **c = **c+20;
        printf("%d", a);
        return 0;
}
```

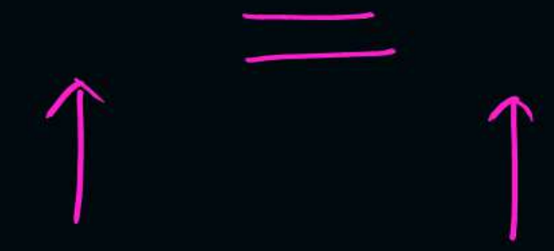


pointer to pointer to an integer

40+20

typecast

int *a = 10



type

```
int *ptr = 10;
```

mismatch

```
int *ptr = (int*)10;
```

type casting

Call by reference

```
#include<stdio.h>
void fun(int *a, int *b){
    *a = *a - 10;
    *b = *b + *a; //
}
int main(){
    int x=10, y=20;
    fun(&x, &y);
    printf("%d %d", x, y);
    return 0;
}
```

a 100 b 200

20+0=20

x 10 0 — 100

y 20 — 200

0, 20

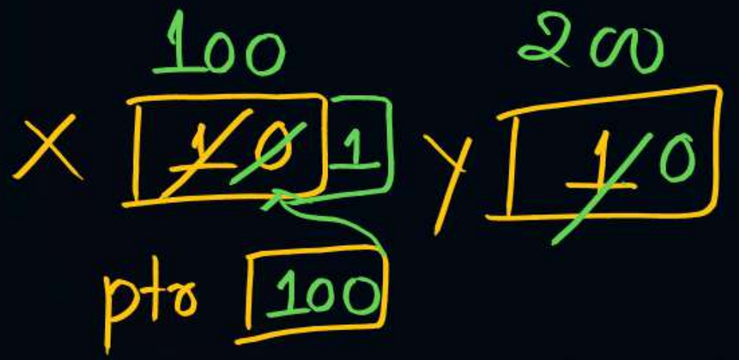


GATE 2017

#Q Consider the following function implemented in C:

```
void printxy (int x, int y) {
        int *ptr ;
        x = 0;
        ptr = &x;
        y = * ptr;
        * ptr = 1;
        printf ("%d, %d," x, y);
}
```

1, 0



The output of invoking printxy (1, 1) is

(A) 0,0

(B) 0,1

(C) 1,0

(D) 1,1

Slide

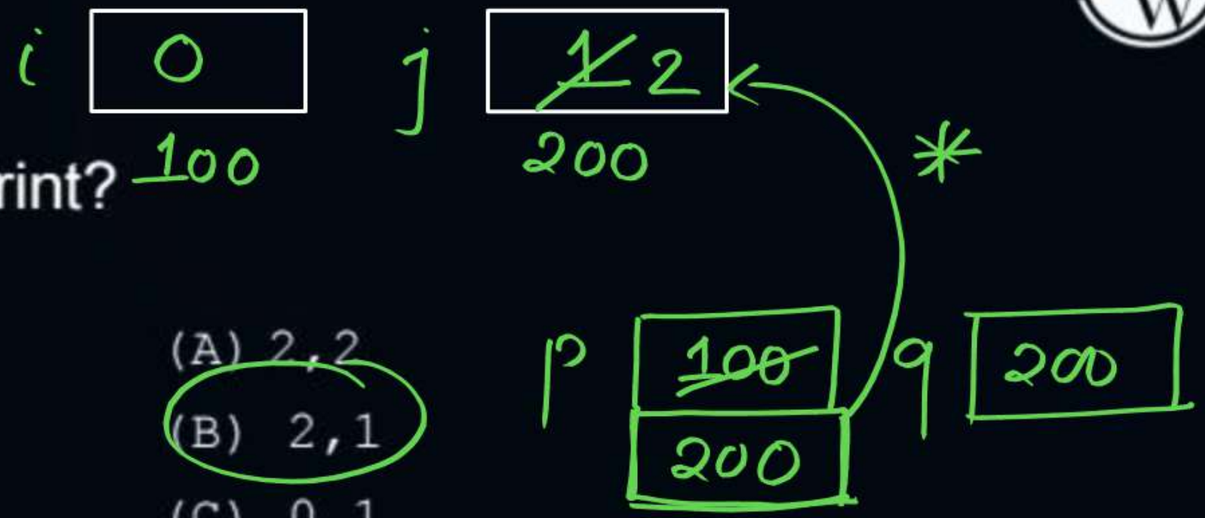


Call by reference

#Q What does the following program print?

```
#include<stdio.h>
void f(int *p, int *q) {
    p=q;
    *p=2;
}
int i=0, j=1;
int main() {
    f(&i, &j);
    printf("%d %d\n", i,j);
    return 0;
}
```

(A) 2,2
(B) 2,1
(C) 0,1
(D) 0,2



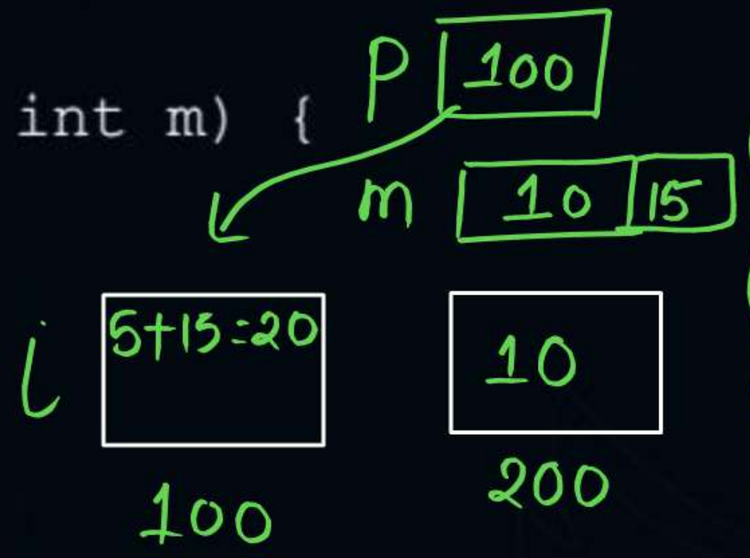


Question

The value printed by the following program is _____.

```
void f (int * p, int m) {
     m = m + 5;
    *p = *p + m;
     return;
}
void main () {
 int i=5, j=10;

 f (&i, j);
 printf ("%d", i+j);
}
```



(A) 25
(B) 30 ✓
(C) 35
(D) None

20+10 = 30

Slide



Question

Consider the following C program?

```
#include <stdio.h>
void func(int *x,int *y,int *z){
    *y=*y+4;
    *z=*x+*y+*z;
}
int main()
{
    int x = 10, y=3;
    func(&y, &x, &x);
    printf("%d %d",x,y );
    return 0;
}
```

The value printed by above program is

(A) 10, 3

(B) 31, 3

(C) 27, 7

(D) 27, 3



2 mins Summary

Topic — Register

Topic — Extern

Topic — pointer

Topic — Call by reference

Topic

Slide

THANK - YOU

